

Here is how to set up the repository module for **Stage 1**, keeping all your database logic encapsulated in one place without touching your routes.

1. Create your Environmental Files

Create a `.env` file in your root folder (ensure this is listed in your `.gitignore` so it is not tracked by Git):

```
ini
```

```
DATABASE_URL=postgres://postgres:dev@localhost:5432/tasks
```

Use code with caution.

Create a `.env.example` file to commit to your repository:

```
ini
```

```
DATABASE_URL=postgres://username:password@localhost:5432/database_name
```

Use code with caution.

2. Node.js Lane (Express with `pg`)

First, install the PostgreSQL driver and environmental variables library:

```
bash
```

```
npm install pg dotenv
```

Use code with caution.

Create a new file named `db.js` to manage your initialization logic and database operations:

```
javascript
```

```
// db.js
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

const initDb = async () => {
  const client = await pool.connect();
  try {
    // 1. Create table if it doesn't exist
    await client.query(`
      CREATE TABLE IF NOT EXISTS tasks (
        id SERIAL PRIMARY KEY,
        title TEXT NOT NULL,
        done BOOLEAN DEFAULT FALSE
      );
    `);

    // 2. Count existing records
    const res = await client.query('SELECT COUNT(*) FROM tasks;');
    const count = parseInt(res.rows[0].count, 10);

    // 3. Seed initial 3 values only if empty
    if (count === 0) {
      const seedQuery = `
        INSERT INTO tasks (title, done) VALUES
        ('Buy groceries', false),
        ('Clean the room', true),
        ('Study API design', false);
      `;
      await client.query(seedQuery);
      console.log('Database initialized and seeded.');
    } else {
      console.log('Database already exists with data. Skipping seed.');
    }
  } catch (err) {
    console.error('Error initializing database:', err);
    throw err;
  } finally {
    client.release();
  }
}
```

```
};

// Initialize connection immediately on module load
initDb();

module.exports = pool;
```

Use code with caution.

3. Python Lane (FastAPI with `psycopg`)

First, install the PostgreSQL driver and environment parser:

bash

```
pip install "psycopg[binary]" python-dotenv
```

Use code with caution.

Create a new file named `database.py` to manage your connection and startup logic:

python

```
# database.py
import os
import psycopg
from psycopg.rows import dict_row
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

def get_db_connection():
    # Returns a connection that maps columns to dictionary structures
    return psycopg.connect(DATABASE_URL, row_factory=dict_row)
```

```
def init_db():
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                # 1. Create table if it doesn't exist
                cur.execute("""
                    CREATE TABLE IF NOT EXISTS tasks (
                        id SERIAL PRIMARY KEY,
                        title TEXT NOT NULL,
                        done BOOLEAN DEFAULT FALSE
                    );
                """)

                # 2. Check if table is empty
                cur.execute("SELECT COUNT(*) FROM tasks;")
                count = cur.fetchone()["count"]

                # 3. Seed initial 3 values if empty
                if count == 0:
                    cur.execute("""
                        INSERT INTO tasks (title, done) VALUES
                        ('Buy groceries', false),
                        ('Clean the room', true),
                        ('Study API design', false);
                    """)
                    conn.commit()
                    print("Database initialized and seeded successfully.")
                else:
                    print("Database already contains records. Skipping seed.")
    except Exception as e:
        print(f"Error connecting or initializing the database: {e}")
        raise e

# Run initialization upon module lookup
init_db()
```

Use code with caution.

Verification Checkpoint

Run your terminal verification commands to verify the tables and rows are set up correctly:

bash

Check that the table was created successfully

```
docker exec -it taskdb psql -U postgres -d tasks -c "\dt"
```

Check that exactly 3 rows are inside the database

```
docker exec -it taskdb psql -U postgres -d tasks -c "SELECT * FROM tasks;"
```